SCC Kernel Fortress Orbital Launch Checklist

Launch-control artifact for the SCC Kernel Fortress release path

Mechanization-hardened 10/10 pass

Launch doctrine. Legality is a runtime invariant. A system that cannot prove the legality of its next step has no right to take it.

Operating rule. If any required gate is red, do not ship. Fix the gate, rerun the transcript, update the release manifest, and only then tag the release.

Current launch status

Current state: Stage 0 is NO-GO. Rust-equipped validation, Docker/CI clean-run evidence, and signed SHA256 release evidence still have to be attached. This is the correct release posture: the package has strong local evidence for Python validators, TypeScript reference tests, formal static checks, vector validation, and Builder Scaffold tests, but it must not claim full v1.0 orbital insertion until Rust, CI, Docker, and signing evidence are green on clean runners.

Stage 0: Pre-launch - current state to ignition

Gate	Evidence required	Current status	Decision
Existing validation transcripts green	Rust, TypeScript, Python scaffold, golden vectors, negative corpus	Partial: TS/Python/vector/formal static green; Rust not observed locally	**NO-GO**
Release manifest verified and signed	SHA256 file, manifest, detached signature, public verification command	SHA256 present; signature not attached	**NO-GO**
Docker and CI gates pass	Fresh runner transcript, artifact upload, CI job URL or exported log	Scripts/workflow present; clean runner not observed here	**NO-GO**
NeurIPS E&D package ready	PDF, checklist, supplement zip, anonymized package	Ready in package	**GO**
Public repo initialized	CC-BY-SA-4.0 LICENSE, README, Verify in 5 Minutes command	Package docs present; public repo not verified here	**NO-GO**

Stage 0 launch criterion. Run `scripts/stage0_prelaunch_gate.sh` on a Rust + Docker equipped clean runner and obtain `stage0_prelaunch=PASS` with SHA256/signature evidence attached.

Stage 1: v1.0 Kernel Fortress - First Orbital Insertion

Goal. Make reviewers say, "this is real." The reviewer must verify the core artifact in under five minutes and see a complete release transcript.

Requirement	Acceptance criterion	Evidence artifact
Rust checker impeccable	`cargo fmt --check`, `cargo clippy --all-targets --all-features -- -D warnings`, full tests, coverage report	`rust gate log`
Negative corpus ≥ 30	Every fixture has fixture, expected first-failure gate, and canon article	`negative manifest`
Golden vectors ≥ 8	Exact, Stutter x2, SafeRefined x2, governance update, recovery, memory compression, long stutter chain	`golden manifest`
Differential testing	Rust == TypeScript == Python on every fixture and first-failure gate	`differential matrix`

Proptest + fuzz	At least one hour, zero failures, seed log retained	`fuzz 1h log`
TCB ledger	Every crate/version justified with replacement plan	`TCB ledger`
Security review	STRIDE model plus mitigation matrix	`STRIDE review`
Documentation	One-pager overview, interactive TS demo, reviewer quickstart video outline/script	`docs/demo/media`
Tag release	CI publishes artifacts automatically on tag	`release workflow`

Stage 1 launch criterion. `bash scripts/run_release_gates.sh` prints `release_gate=PASS`, CI publishes release artifacts on a tag, and the transcript contains no missing Rust, Docker, fixture, signing, or coverage evidence.

Stage 2: v2.0 Mechanized Beast - Second Stage Burn

Goal. Turn executable evidence into formally grounded evidence.

Requirement	Acceptance criterion	Evidence artifact
Lean 4 mechanization	Checker soundness, PO bundle reflection, Step Simulation Law; at least 70 percent of critical path	`Lean files`
Rust verification fragment	Partial Prusti/Kani verification on checker core	`rust contracts`
Numerical stability report	Error bounds for simplex projection, drift, risk vectors	`numerical TCB`
Federation expansion	Quorum safety gates and cross-shard lineage tests	`federation/`
Memory compression proofs	Entropy monotonicity proof notes and reconstruction tests	`memory-lineage stress doc`
Public corpus	Negative corpus repo and fixture classification explorer	`negative corpus`
Certification templates	Compliance self-assessment for adopters	`certification/`
Independent ports	Zig and Go reference implementations pass differential tests	`ports/`

Stage 2 launch criterion. Mechanization paper draft exists, proof-equipped gates run green, and all new fixtures pass Rust/TS/Python differential testing.

Stage 3: v3.0 Ecosystem Domination - Third Stage and Orbital Refueling

Goal. Make SCC a practical standard that external teams can adopt.

Requirement	Acceptance criterion	Evidence artifact
Full proof direction	Lean/Coq checker soundness and extraction path to verified Rust fragments	`formal/`
Production-grade ports	Rust FFI, WASM, embedded profile	`ports/`
BOTG adversarial suite	10k+ step traces, gradient-hacking attempts, Byzantine shard scenarios	`adversarial/`
Treaty and amendment framework	Live governance simulation	`governance_sim/`
Third-party audit	Published independent security review	`audits/`
Certification program	Badge criteria and public registry	`certification/`
Conference path	NeurIPS E&D plus systems/security submissions	`papers/`
Community	Discord/forum, roadmap, contribution guide, SCC in 100 lines	`community docs`

Stage 3 launch criterion. Multiple external teams can run the framework, pass the certification checklist, and reproduce the evidence without private context.

Stage 4+: Constellation Mode - We Keep Firing

- Continuous fuzzing and nightly regression runs.
- Live governance on a reference agent.
- Integration with major agent frameworks.
- Formal verification of the full seven-stage pipeline.

- Global negative-corpus crowdsourcing.
- Annual SCC Fortress Day with new fixtures and adversarial challenges.

10/10 success metrics

- A reviewer can verify the artifact claim in under five minutes.
- There are zero open issues on the critical path.
- A downstream project can pass certification without private help.
- The TCB is smaller, clearer, or better justified every release.
- The repo feels complete enough that a tired reviewer understands the evidence in one pass.

Final countdown command

Before every push, answer all five:

1. Does this change preserve protected isolation?
2. Does the checker still reject every negative fixture at the named first-failure gate?
3. Would a tired reviewer understand the evidence in one pass?
4. Is the TCB smaller or better justified?
5. Are we closer to mechanized soundness?

If yes to all five, light the engines.